

# Powering Up Your Types with Annotated

7/14/2026, 6:40:01 PM

1/48

Hey everyone – first talk at EuroPython, really glad to be here.

Today we're looking at typing.Annotated – introduced in Python 3.9, as a result of PEP 593 (and how long ago was that, right?)

It's one of the most powerful features in Python's type system, but in my view it's still somewhat underused.

We'll walk through how it works, what the ecosystem has built on top of it, and write real code along the way. Let's get into it.

---

## 2/48 Who am I

Before we dig in, let me start with a quick introduction.

Vlad Fedoriuk – software engineer at ActiveCampaign, five-plus years with Python. And as I said – first talk at EuroPython, not nervous at all. (I'm lying.)

We also have a booth in the hallway, so come say hi after.

Alright – let's talk types.

---

### 3/48 Two distinct usage patterns

The way I see it, `Annotated` has two main patterns.

Semantic types – naming what your data actually is, not just its Python type.

Framework metadata – attaching instructions that tools read at runtime.

Next slides show what that looks like.

---

### 4/48 Use case: semantic types

Imagine you have a replicated database setup with separate reader and writer instances. You need to annotate an engine parameter in a function, but plain `Engine` doesn't tell you which one you're using.

`Annotated` fixes that. `ReaderEngine` and `WriterEngine` are the same runtime type with different labels. The type checker sees `Engine`; the code reviewer sees the intention.

The metadata is just a string – it could be anything.

---

### 5/48 Use case: framework metadata

Here we're attaching instructions for external libraries to read.

`Pydantic` checks if `total` is at least zero, and `FastAPI` expects the query parameter to be at least three characters.

These libraries check both the types and the metadata – static typing only handles the integer and string checks, while runtime libraries additionally read the annotations to validate or even document our API.

---

### 6/48 `Annotated` 101

Now that we've seen why we want this metadata, let's look at how it works under the hood.

We'll cover the two main attributes python gives us, and how nesting is resolved.

---

## 7/48 `<span class="slide-title-code">\_\_origin\_\_</span>` and `<span class="slide-title-code">\_\_metadata\_\_</span>`

Every Annotated type has two main attributes: dunder `__origin__` and dunder `__metadata__`.

Dunder `__origin__` gives you the base type – string in this case.

Dunder `__metadata__` is a tuple that keeps all constraints in the order you defined them.

One quick catch: the helper function `get_origin()` returns `Annotated` itself, not the base type. To get the base type, read dunder `__origin__` directly.

---

## 8/48 Nesting flattens

When you nest Annotated directly, Python automatically flattens it.

The metadata from the inner type comes first, followed by the outer one. This means they combine into a single tuple, and type comparison works exactly as you'd expect.

But this only happens when you nest them directly. Next, we'll see where this breaks.

---

## 9/48 Aliases block flattening

When you put Annotated behind a type alias, flattening stops working.

This is because these type aliases are evaluated lazily. The outer Annotated sees only one opaque object, hiding the inner metadata and breaking direct type comparison.

Because of this, library authors have to explicitly evaluate aliases to find those hidden constraints.

---

## 10/48 `Annotated` is not a type

Another key thing to understand: `Annotated` is not a type. It's a "special form" – closer to a syntax keyword than a concrete Python class.

At runtime, wrapping something in `Annotated` produces a private `_AnnotatedAlias` object. That means it fails `isinstance` checks against `type`.

Because of this, if a library expects a `type`, passing an `Annotated` type will cause type checkers to reject it.

---

## 11/48 The problem

This is a case that a popular library stumbled upon in real life.

The service locator library `svcs` recommended using `Annotated` types as registry keys. It worked fine at runtime, but type checkers started raising errors because the registration function expected a type object.

Since `Annotated` is a special form, type checkers were right to complain. Let's see how we can fix this.

---

## 12/48 The fix – `TypeForm`

A possible solution here is `TypeForm`, introduced in PEP 747 and landing in Python 3.15.

While a type object as type annotation only accepts concrete classes, `TypeForm` accepts any valid type expression, including `Annotated` types.

 1 Major type checkers have started supporting this experimentally today. If you have a similar use case where you need to pass type expressions around, `TypeForm` is the tool you might want to look at.

---

## 13/48 Consuming metadata at runtime

Now let's talk about the other side: how do we actually read these annotations when our code runs?

We'll look at the built-in tools Python gives us, and some common patterns you'll see.

---

## 14/48 Example classes

To demonstrate these runtime tools, we'll use these simple classes.

The first class has a `name` attribute, and the `Product` class inherits from it, adding `price` and an optional `discount`.

Notice that only `price` and `discount` are defined directly on the `Product` class. This distinction will be important in the next few slides.

---

## 15/48 Read annotations with the API

To read annotations, avoid accessing the raw `__annotations__` dictionary directly, as it has inconsistent behaviors on older Python versions.

Instead, use `annotationlib.get_annotations` in Python 3.14+, or `inspect.get_annotations` in Python 3.10+.

Keep in mind that these APIs only return the annotations defined directly on the class itself – they do not walk the MRO to merge parent class annotations.

---

## 16/48 `annotationlib.get_annotations`

The new `annotationlib` library API lets you retrieve annotations in three different formats.

The `value` format gives you fully evaluated Python objects.

The `string` format returns the raw source text as written.

The `forward-ref` format returns proxy objects instead of raising a `NameError` if a type is not defined yet.

This is highly useful for library code that needs to inspect types before they are fully declared.

---

## 17/48 `typing.get_type_hints`

For general application code, the `get_type_hints` function is often what you'll want instead.

Unlike the low-level `get_annotations` API, it automatically traverses the `MRO` to collect inherited types and resolves all forward references.

One important detail: by default, it strips `Annotated` wrappers away. You have to pass the `include_extras` flag to keep the metadata intact.

---

## 18/48 `annotated-types`

Now let's look at `annotated-types`.

This is the library that provides a common set of constraints that different libraries can share. Let's see how it works.

---

## 19/48 Shared constraint vocabulary.

The library `annotated-types` was created at PyCon 2022 by Samuel Colvin, the Pydantic maintainer, alongside the Hypothesis team.

The goal was a shared library of constraints, so other libraries wouldn't have to reinvent definitions like "greater than" or "minimum length".

It gives the community a unified vocabulary, which we'll look at next.

---

## 20/48 `BaseMetadata` + `GroupedMetadata`

The library defines two simple contracts: `BaseMetadata` and `GroupedMetadata` .

`BaseMetadata` is the base class for simple constraints, and `GroupedMetadata` is a protocol for composite ones, like `Interval` which yields `Gt` and `Lt` .

These contracts let any library safely scan type annotations without having to import or know about specific third-party types they don't own.

---

## 21/48 The `get_metadata` idiom

This is the recommended helper function to extract those constraints at runtime.

- 1 We verify that the type is indeed `Annotated` ,
  - 2 and then iterate through its arguments. We yield simple constraints directly
  - 3 and unpack composite ones using the `GroupedMetadata` protocol.
  - 4 This is basically the blueprint libraries like Pydantic or Hypothesis use under the hood.
- 

## 22/48 Toy `Dependency Injection` Auto-wiring

Next, we're going to build a toy auto-wiring tool in about twenty lines of code.

It will help us see how the exact same metadata patterns we've been looking at can be used to resolve dependencies at runtime.

---

## 23/48 Auto-wiring with `Annotated`

What if `annotated-types` shipped `Depends` as a primitive?

1 We define a simple `Depends` dataclass that inherits from `BaseMetadata`.

2 Then, the application's code would consume it like this – annotate function parameters with it, like telling `current_user` to get its `env` string from `settings`.

3 Next, we need the function that actually resolves these dependencies at runtime.

---

## 24/48 Filter constraints for `Depends`

This is exactly what a third-party package consuming `annotated-types` would have to do – scan the annotations, find the metadata it cares about, and act on it.

1 It uses `get_type_hints` with `include_extras` set to `True` to inspect the parameters.

2 We look for `Annotated` types

3 find the first `Depends` metadata instance, and recursively resolve its callable.

4 This gives us a dictionary of resolved dependencies that we can pass directly to our target function.

---

## 25/48 Call with injected deps

This is what it looks like in practice. We have three simple functions, with dependencies chain-linked to one another.

We run the resolver on the endpoint. Our code walks the tree, runs `config`, then the database URL function, and resolves the final arguments.

We get our injected parameters as keyword arguments, ready to call the endpoint.

---

## 26/48 What the toy leaves out

While our twenty-line tool works, production-grade engines solve a few more engineering problems.

✎1 First: Deduplication and Caching. We don't want to re-resolve the same dependency multiple times within a single request.

✎2 Second: Async and Generators. We need to handle async functions and yield-based setup and teardown.

✎3 Third: Scoping. We must isolate state per call so concurrent requests don't leak into each other.

✎4 And fourth: Overriding. We need to easily swap dependencies in our tests.

Now, let's look at the libraries in the ecosystem that handle this.

---

## 27/48 The DI ecosystem

The Python dependency injection landscape has a lot of options, ranging from older, established packages to very fresh releases.

Personally, `svcs` is my favorite. Another one definitely worth your attention is `uncalled-for` – it's battle-tested inside FastMCP, but is also super minimalistic and simple in its core.

You have plenty of great options to choose from depending on what fits your project.

Let's move from theory to writing some real code.

---

## 28/48 Let's build something

Now, let's build something concrete.

We'll design a room reservation system from the ground up, starting with simple Python dataclasses and layering metadata as we go.

---

## 29/48 Room Reservation

We start with simple Python dataclasses. No databases or frameworks yet.

We have a `Reservation` and a `Room`. The `Room` checks rules like room capacity and date overlaps when we add a new reservation.

Right now, everything uses plain primitives like `str`, `int`, and `Decimal`. Let's change that.

---

## 30/48 Make your types mean something

We swap the raw primitives for Annotated aliases.

1 `RoomId` is still a string under the hood, and `GuestCount` is still an integer. But now, constraints like minimum length and value ranges live directly inside the type declarations.

This keeps the constraints right next to the specific domain types they describe.

When we say something is a `RoomRate`, we're not just saying it's a decimal – we're saying exactly what kind of decimal it is.

---

## 31/48 Polyfactory: almost zero config

Now let's generate test data. Polyfactory reads `Ge`, `Le`, `Gt`, `MinLen`, and `MaxLen` straight from the Annotated metadata – no overrides needed for those fields.

1 Two fields still need help. `ends_at` has a cross-field dependency – it has to land after `starts_at` – so we use `PostGenerated` to compute it once `starts_at` is already resolved.

2 `created_at` hits an actual polyfactory bug: our `Timezone(...)` constraint means "any timezone is fine," but polyfactory tries to pass that ellipsis straight in as the `tzinfo` argument, which blows up. The workaround is a plain `Use()` with an explicit `UTC`.

3 Predicate constraints get silently skipped. Polyfactory has no way to guess how to satisfy an arbitrary callable.

---

## 32/48 Using factories in tests

Now we actually use the factory in a test.

1 `ReservationFactory` spins up `guest_count`, `rate`, and `room_id` straight from the Annotated constraints.

2 We hand that reservation to `RoomFactory` as its starting state

3 then add the exact same reservation again – same `starts_at`, same `ends_at` – which trips the overlap check against itself.

---

## 33/48 Hypothesis: property-based testing

The factory test was example-based – one scenario, one assertion. Property-based testing flips that: describe properties that should always hold, and Hypothesis generates hundreds of random inputs to verify them.

1 `st.builds` reads the dataclass and generates every field from their Annotated constraints – `guest_count`, `room_id`, `starts_at`, `ends_at`, all automatic.

2 `st.from_type` does the same for individual types. `Percentage` and `TaxRate` both have Annotated constraints, so Hypothesis knows what valid values look like.

3 One caveat – Hypothesis's Annotated support is still basic. Nested aliases can fail to resolve, and constraints like `Timezone` or `IsFinite` aren't well supported yet. You might still write a strategy by hand sometimes.

---

## 34/48 </span>SQLAlchemy</span>: reusable column blueprints

Testing's done, let's move on to persistence. Here are the same aliases we defined earlier – still just the type and its constraint, nothing SQL yet.

1 Now we add the ORM config to the alias – SQL type, server defaults – since they're always true for this type.

2 Here's the model using those aliases. Nullability, uniqueness, and indexes are all table decisions, so they land on the field, and SQLAlchemy merges both column configs together – alias for the type, field for the table.

And `mapped_as_dataclass` decorator is just one way to wire this up – a classic declarative base subclass or the dataclass base class work just as well with the same aliases.

---

## 35/48 Generating migrations

Once the models are defined, generating the migration is the easy part – Alembic scans them and writes it for us. Column types, server defaults, indexes – all of it comes straight from the Annotated aliases.

This is trimmed from the actual generated output.

---

## 36/48 </span>Pydantic</span> & </span>FastAPI</span>: composable schemas

Now for the API side. Same aliases, new reader – Pydantic validates the incoming data, and FastAPI turns it straight into OpenAPI docs.

RoomId, GuestCount, StayDate, and RoomRate are exactly the same aliases we declared before. Pydantic reads the annotated-types constraints, MinLen, Ge, Le, Gt, straight off the type and uses them to parse and validate incoming data.

1 Need API-only stuff, like a title on room\_id here? Wrap it in Annotated and stack a Field.

---

## 37/48 Validation & serialization as metadata

Pydantic ships its own functional validators and serializers, and they just slot into Annotated right alongside Gt or MinLen.

- 1 The `wrapValidator` rounds the Decimal to two places after Pydantic validates it, and the `PlainSerializer` turns it into a float for both Python and JSON output.
- 

## 38/48 Fits together with `FastAPI`

RoomId shows up here too, now as a FastAPI path parameter.

- 1 The annotated room ID with a path title is basically what we just did with Field, just for path parameters this time.

- 2 DepContainer is Annotated too – it's the dependency injection from earlier.

- 3 So, where does it bring us from here?
- 

## 39/48 Nest. Overlay. Compose.

Three verbs sum up the pattern:

- nest types inside Annotated,
- overlay metadata,
- and compose richer meaning without touching the type itself.

This is an approach I've been using for some time now, and it's proven to be really pleasant to work with.

Let's explore this deeper.

---

## 40/48 Orthogonal metadata

One RoomId, three concerns stacked in the same alias – SQLAlchemy, Pydantic, and FastAPI each hook into the piece they understand and skip the rest.

👉1 The type stays central – stack as many concerns as you need, and neither the database nor the API gets the final say.

👉2 Types stop being just annotations for the type checker – they become central to your data model, defining what the data actually is.

👉3 And when the ecosystem agrees on shared standards like annotated-types, that composition goes even further – these contracts work across libraries without anyone coordinating.

---

## 41/48 Why reinvent the wheel?

There are already frameworks solving this same problem – let's see what they actually do, and where the pressure starts to build.

This isn't about replacing them. It's about knowing when Annotated gives you something they can't.

---

## 42/48 The Django way

Django uses the Active Record pattern – the database table is the one model that dictates everything.

Change a field, and that ripples through the serializer, the validation, the views. There's no separate layer holding your business rules – just the database, pushing pressure onto everything above it.

---

## 43/48 The SQLAlchemy way

SQLModel merges Pydantic and SQLAlchemy into one class – great for simple CRUD, but it ties your database table to your API schema.

The moment your write schema needs to hide a field or shape data differently than the read schema, you're splitting classes again.

---

## 44/48 Structural pressure

This is design pressure – when your database structure and your API contract keep pulling on each other. Hynek's got a whole talk on it, linked right there.

1 Change a column, and it ripples straight through your endpoint – one side always ends up in charge.

2 None of this is bad. For CRUD apps, stable schemas, internal tools, Django and SQLAlchemy shine exactly because of this coupling.

3 And sometimes it's fine to repeat yourself – duplicate first, refactor once you actually understand the shape you need.

---

## 45/48 One more layer

We've covered a lot – Annotated internals, type aliases, dependency injection, metadata layering. One more layer before we wrap up: documentation.

---

## 46/48 Documentation: `annotated-doc`

For our last layer, documentation. `annotated-doc` adds a `Doc` object right inside the type. It's built on the revoked PEP 727, championed by Sebastian Ramirez, and already powers Typer and FastAPI's docs.

`Doc` has to be the first item in the annotation, and applied directly to the constant or argument, not through an alias – otherwise doc tools might not pick it up correctly.

---

## 47/48 MkDocs / Zensical

This extension reads `Doc` metadata from your Annotated types and renders it in the docs. No need for docstring styles like Numpy or Google, and no duplication.

Add or remove a field, and the docs update automatically.

---

## 48/48 Thank you!

Thanks for sticking with me through all this. We saw Annotated work as a composable metadata engine – one type, read differently by validation, persistence, and documentation, without any of them depending on each other.

Happy to take any questions.